A PRACTICAL EXPERIENCE WITH AI

AI and the Future of Software Development

Is AI replacing software developers, or is it changing who can build software?

I had limited practical experience with React and JavaScript. With AI assistance, I built and connected a working frontend application.

A personal reflection on AI-assisted development, the new accessibility of software creation, and the real risks for developers.

By a backend developer learning while building

01 / THE BIG QUESTION

Is AI replacing developers?

This is no longer a theoretical discussion. AI can generate code, explain unfamiliar technologies, suggest the next step, identify common errors, create testing guidance, design interfaces, and connect applications to APIs.

The more important question may be this:

Is AI replacing software developers, or is it changing who can participate in software development?

The opportunity

People with ideas and domain knowledge can build working software more quickly.

The disruption

The time, knowledge, and team size traditionally required to create software are shrinking.

This document is based on a practical experience rather than a prediction about the future.

02 / MY STARTING POINT

I started outside my comfort zone

I am primarily a backend developer. Before this experience, I had limited practical knowledge of React, JavaScript, frontend architecture, component-based development, client-side routing, and frontend API integration.

I did not wait until I had mastered frontend development before beginning. I used AI as a development partner and learned while building.

What I already understood

Backend APIs, business rules, authentication requirements, database behavior, and the product I wanted to build.

What I needed to learn

How the frontend was structured, how components communicated, how requests moved, and how the interface connected to the backend.

The old sequence was often: learn first, build later. My sequence became: start building, then learn in context.

03 / WHAT I BUILT

From backend requirements to a working frontend

With AI guidance, I developed a functional frontend for an Academy Management System.

Authentication

Login, tokens, protected requests, session restoration, password change, and password reset.

Application shell

Dashboard navigation, responsive layout, sidebar, top bar, professional forms, and feedback states.

Management features

Academies, users, role and permission management, endpoint-level access, and academy-scoped data.

Integration

Frontend requests connected to existing backend APIs with success, validation, unauthorized, forbidden, and error messages.

A person with limited frontend experience can now move from an existing backend system to a usable interface with AI assistance.

04 / THE NEW WORKFLOW

AI changed how I moved forward

The process became more direct and iterative:

1	**Explain** Describe the requirement, API behavior, and desired result.
2	**Generate** Create a project structure, component, page, or integration.
3	**Run** Open the application and observe what actually happens.
4	**Share** Provide the error, confusing behavior, or missing requirement.
5	**Improve** Apply the next fix, test it, and continue building.

AI did not require me to discover every solution alone. It helped me move from one development step to the next.

05 / THE BARRIER IS LOWER

Start before mastering every technology

My experience showed me that AI can dramatically reduce the barrier to software development.

Start earlier

Begin with a clear idea instead of waiting until every technology is familiar.

Learn in context

Understand files, components, requests, and errors while working on a real product.

Build more

Generate components, API calls, authentication flows, navigation, and testing guidance.

Move faster

Reduce the time spent searching for basic direction and common debugging steps.

A person with domain knowledge, a clear idea, persistence, and access to AI can now build much more than before.

06 / BEFORE FULL UNDERSTANDING

AI can provide a path forward

One of the most important changes is that AI can sometimes provide a useful solution before the developer fully understands the problem.

It can explain:

Project flow

Which file is responsible, which component calls another, and how a request moves through the application.

Next action

What to change, what to run, what to test, and what information is needed when an error appears.

This changes the learning process. Instead of learning everything first and building later, people can begin building and learn in context.

That is not a small convenience. It changes who can enter the field and how quickly they can produce a first version.

07 / THE RISK IS REAL

AI is changing the economics of development

AI is not only making experienced developers faster. It is also allowing people from one technical area to work in another area more quickly.

Smaller teams

Companies may produce more software with fewer people.

Entry-level pressure

Some basic implementation and repetitive tasks may be automated.

Higher expectations

Clients may expect faster delivery at lower cost.

More competition

A single person may perform work that previously required multiple specialists.

The risk is not imaginary. AI is already changing the amount of time and knowledge required to produce software.

08 / WHAT AI DOES NOT GUARANTEE

Generated code is not automatically reliable

AI can create software, but the output can still contain serious problems.

Technical risks

Security vulnerabilities, poor architecture, performance issues, incorrect API integration, and hidden maintenance costs.

Product risks

Incorrect business assumptions, incomplete validation, poor data handling, and a solution that does not match the real user need.

The difference is not that these risks disappear. The difference is that AI can now help with many activities that previously slowed development down.

Someone still needs to decide whether the solution is appropriate. Someone must take responsibility for the final product.

09 / THE ROLE IS CHANGING

The developer is becoming more than a code writer

The future developer may spend less time typing repetitive code and more time making decisions that determine whether the product succeeds.

Human strengths

Business understanding, product thinking, communication, problem definition, and ownership.

Technical strengths

System design, security awareness, testing, data protection, review, and maintenance.

The question may no longer be:

“Can you write this code?”

It may become:

“Can you use the available tools to create the right solution safely and reliably?”

10 / MY PERSONAL LESSON

I was able to start before I had mastered everything

This experience changed my view of software development.

AI allowed me to build in an area where I had very little previous knowledge. It helped me understand project structure, follow the request flow, connect frontend and backend systems, solve errors, and continue moving forward.

I did not become an expert frontend developer overnight.

But I was able to start before I had mastered everything.

That is the major change: AI made it possible to learn and build at the same time.

The opportunity

More people can turn ideas into prototypes and products.

The responsibility

People must still understand the goal, review the result, and own the consequences.

THE BIGGER QUESTION

AI is changing who can build software.

It is reducing the time, knowledge, and resources traditionally required to create software.

The threat is real for repetitive work and some entry-level tasks. The opportunity is real for people who learn to use AI effectively and take responsibility for what they build.

The future may not belong only to people who write every line of code manually. It may belong to people who can understand a real problem, explain it clearly, use AI to accelerate development, review the result, protect the system, test the product, and deliver something useful.

Is AI replacing software developers, or is it creating a new type of developer?

What has your practical experience been? Share your honest opinion, concerns, or a different view of the future.

A practical reflection on AI-assisted software development